

ADVANCED SQL

1.

Training Dataset

Use the same dataset from the core SQL module

Table	Grain	Purpose
customers	1 row = 1 customer	Customer identity, state, signup date, email
orders	1 row = 1 order	Order header, customer, date, status, channel
order_items	1 row = 1 order line	Product and quantity for each order line
products	1 row = 1 product	Product category and unit price

Core relationships: customers → orders, orders → order_items, and products → order_items are one-to-many relationships.

Advanced Data Modeling

Data modeling defines how data is stored, organized, and related. Good modeling reduces unnecessary redundancy, improves consistency, and makes querying and updates more reliable.

Key / Constraint	Purpose
PRIMARY KEY	Uniquely identifies a row and cannot be NULL.
FOREIGN KEY	Defines a relationship to a key in another table and supports referential integrity.
COMPOSITE KEY	Uses multiple columns together as the row identifier.
UNIQUE	Prevents duplicate values in the constrained column(s).
NOT NULL	Requires a value.
CHECK	Requires values to meet a defined condition.
DEFAULT	Supplies a value when one is not provided.

```
CREATE TABLE customers (  
    customer_id INT PRIMARY KEY,  
    full_name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE  
);
```

```
CREATE TABLE orders (  
    order_id INT PRIMARY KEY,  
    customer_id INT,  
    order_date DATE NOT NULL,
```

```

        FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
    );

```

Exercises — Modeling, Keys, and Integrity

1. Identify the grain and candidate primary key for all four training tables.
2. Explain why (order_id, line_no) is a suitable composite key for order_items.
3. Write a query that detects duplicate product_id values in products.
4. Write a query that detects orders whose customer_id does not exist in customers.
5. Add a CHECK constraint conceptually requiring quantity > 0. Explain what bad data it prevents.
6. Explain the difference between entity integrity and referential integrity using the training dataset.

6. Normalization and Denormalization

Normalization separates data into related tables to reduce redundancy. Denormalization intentionally combines or duplicates selected data to simplify and accelerate analytical reads.

Form / Pattern	Goal
1NF	Atomic values; no repeating groups.
2NF	Remove partial dependency on part of a composite key.
3NF	Remove transitive dependency; non-key attributes depend on the key.
Denormalized analytical table	Reduce joins and simplify read-heavy analytics.

In OLTP-style systems, normalization is generally useful for consistency. In analytics and warehouse marts, denormalized fact/dimension or wide-table patterns are often used for read efficiency and usability.

Exercises — Normalization

7. Given a customer table containing order1, order2, and order3 columns, redesign it into normalized tables.
8. Explain why storing product_name and unit_price repeatedly in every order header can create update anomalies.
9. Identify which current training tables behave like transactional entities and which behave like reference data.
10. Design a denormalized order_enriched table and state its intended grain.
11. Explain one benefit and one risk of denormalizing customers, orders, and order revenue into a single mart.

7. CTEs and Advanced Subqueries

This section extends the core SQL module. CTEs make multi-step transformations easier to read, test, and maintain. Subqueries are useful when a query depends on an intermediate scalar, list, or derived result.

```

WITH order_revenue AS (
    SELECT
        oi.order_id,
        SUM(oi.quantity * p.unit_price) AS revenue
    FROM order_items oi
    JOIN products p
        ON oi.product_id = p.product_id
    GROUP BY oi.order_id
)
SELECT

```

```

        o.customer_id,
        SUM(COALESCE(r.revenue, 0)) AS lifetime_revenue
FROM orders o
LEFT JOIN order_revenue r
    ON o.order_id = r.order_id
WHERE o.status = 'COMPLETED'
GROUP BY o.customer_id;

```

break complicated transformations into named, testable steps. Always verify the grain of every CTE before joining it downstream.

Exercises — CTEs and Subqueries

12. Use a CTE to calculate revenue per order, then return the top 5 completed orders.
13. Use two CTEs to calculate completed revenue per customer and then rank customers by revenue.
14. Find customers whose lifetime revenue is above the average customer lifetime revenue.
15. Find products whose unit_price is above the average price of their own category.
16. Use a CTE to identify customers with both WEB and MOBILE orders.
17. Build a multi-step query that returns state, customer count, completed orders, and completed revenue.

8. Window Functions and Analytical SQL

Window functions perform calculations across related rows without collapsing the result set like GROUP BY. They are essential for ranking, running metrics, previous/next-row comparisons, deduplication, and analytical transformations.

Function	Typical Use
ROW_NUMBER()	Unique sequence; deduplication; latest record
RANK()	Ranking with gaps
DENSE_RANK()	Ranking without gaps
NTILE(n)	Buckets / quartiles / percentiles
LAG()	Previous row comparison
LEAD()	Next row comparison
SUM() OVER	Running totals
AVG() OVER	Moving averages

```

SELECT
    customer_id,
    order_date,
    order_id,
    ROW_NUMBER() OVER (
        PARTITION BY customer_id
        ORDER BY order_date DESC, order_id DESC
    ) AS rn
FROM orders;

SELECT
    customer_id,
    order_date,
    LAG(order_date) OVER (
        PARTITION BY customer_id

```

```

ORDER BY order_date
) AS previous_order_date
FROM orders;

```

Exercises — Window Functions

18. Rank customers by completed lifetime revenue.
19. Return the top 3 customers by revenue within each state.
20. Rank products by completed revenue within each category.
21. Calculate running completed revenue by order date.
22. Calculate a rolling average using an appropriate ROWS BETWEEN window frame.
23. Use LAG() to show each customer's previous order date.
24. Use LEAD() to show each customer's next order date.
25. Return only the most recent order for every customer using ROW_NUMBER().
26. Use NTILE(4) to divide customers into four lifetime-revenue buckets.
27. Explain the difference between ROW_NUMBER(), RANK(), and DENSE_RANK() using tied values.

9. Data Engineering SQL Patterns

Pattern	Data Engineering Use
Deduplication	Keep the latest or preferred record from duplicate source data.
Reconciliation	Compare source and target counts, sums, keys, and exceptions.
Data quality	Detect NULL keys, duplicates, orphan records, invalid values.
Incremental filtering	Process only new/changed records based on a watermark or timestamp.
Change detection	Compare current and previous values with LAG or joins.
Top-N / latest-N	Select the most important or most recent rows within a business group.
Mart transformation	Change transactional grain into business KPI grain.

Exercises — DE Patterns

28. Create a duplicate version of selected customer records and deduplicate it using ROW_NUMBER().
29. Write source-vs-target row-count reconciliation SQL for orders and order_enriched.
30. Create a DQ query for NULL customer IDs, orphan orders, and orders without order items.
31. Write an incremental filter that selects orders after a supplied last_processed_date.
32. Detect customers whose order status changed between consecutive orders using LAG().
33. Create a customer-level mart with total orders, completed orders, cancelled orders, lifetime revenue, first order date, and last order date.
34. Find customers who have orders but zero completed revenue.
35. Find products that have never been sold.

10. Indexing and Query Optimization

Indexes can reduce data retrieval work, but they also introduce storage and write-maintenance cost.

Index Type	Typical Purpose
Single-column	Lookup/filter on one column
Composite	Queries filtering/sorting by a column combination

Unique	Uniqueness enforcement plus lookup
Clustered / physical ordering concept	Storage ordering behavior depends on database engine
Non-clustered / secondary	Separate lookup structure pointing to rows

```
CREATE INDEX idx_orders_customer
ON orders(customer_id);
```

```
CREATE INDEX idx_orders_status_date
ON orders(status, order_date);
```

```
EXPLAIN
SELECT order_id, customer_id, order_date
FROM orders
WHERE status = 'COMPLETED'
      AND order_date >= '2025-04-01';
```

Optimization checklist:

- Select only required columns rather than defaulting to SELECT *.
- Filter early where logically possible.
- Join on correct keys and verify uniqueness on the expected 'one' side.
- Avoid unnecessary DISTINCT and GROUP BY used only to hide join mistakes.
- Avoid wrapping indexed filter columns in functions when a sargable range predicate is available.
- Use EXPLAIN/query-plan tools to identify scans, index usage, row estimates, and costly operations.

Exercises — Optimization

36. Compare SELECT * with a projection containing only required columns. Explain the practical difference.
37. Rewrite a YEAR(order_date)=2025 predicate using a date range.
38. Create an index appropriate for repeated customer-order lookups and explain the choice.
39. Create a composite-index proposal for status + order_date filtering. Explain why column order matters conceptually.
40. Run EXPLAIN on an unfiltered orders query and on a selective filtered query. Record the differences shown by your SQL engine.
41. Identify a query where DISTINCT is masking duplicate rows caused by an incorrect join, then correct the join.

11. Partitioning for Large Tables

Partitioning horizontally divides a large table into manageable segments such as date, region, or tenant. The main analytical benefit is partition pruning: the engine can avoid reading partitions that cannot satisfy the filter.

- Choose partition columns based on common filters and data distribution.
- Avoid excessively granular partitioning that creates many tiny partitions.
- Date-based partitioning is common for event and transaction data.
- Partitioning is not a replacement for correct query design or indexing/clustering strategies; behavior is platform-specific.

Exercises — Partitioning

42. For a billion-row events table with `user_id`, `event_type`, and `event_timestamp`, propose a partition strategy and justify it.
43. For an orders table queried mostly by month, propose a partition key.
44. Explain partition pruning using an April 2025 date-range query.
45. Give one example of a poor partition column and explain why it would create inefficient partitions.
46. Compare the conceptual roles of partitioning and indexing.

12. Advanced Integrated Exercises

47. Return the latest completed order for each customer.
48. Return the second-highest-revenue customer in each state.
49. Return the top 2 products by completed revenue within each category.
50. Calculate each customer's completed lifetime revenue and percentage contribution to total completed revenue.
51. Calculate running revenue by date and channel.
52. Calculate the difference between each completed order's revenue and the customer's previous completed order revenue.
53. Find customers whose most recent order is `CANCELLED` or `PENDING`.
54. Find customers with three or more orders and rank them by completed-order rate.
55. Create a monthly KPI result containing revenue, order count, customer count, average order value, and cancellation rate.
56. Detect any referential-integrity issues across all four tables.
57. Create a reusable CTE pipeline that produces one row per order before producing one row per customer.
58. Demonstrate a join-multiplication error and then correct it.

13. Final End-to-End Lab

Scenario: The analytics team needs a reliable customer-performance mart built from the transactional training dataset. The solution must preserve correct grain, handle missing relationships safely, calculate analytical metrics, and include validation.

Required Build

- `order_enriched` — one row per order with customer attributes, total units, and order revenue.
- `customer_performance_mart` — one row per customer with order and revenue KPIs.
- `product_category_performance` — category/product metrics with product ranking.
- `dq_metrics` — data-quality and reconciliation metrics.

Required Advanced Logic

- Use at least two CTEs.
- Use `ROW_NUMBER()` for latest-record logic or deduplication.
- Use `RANK()` or `DENSE_RANK()` for a Top-N business requirement.
- Use `LAG()` or `LEAD()` for change/sequence analysis.
- Use a running `SUM()` or moving `AVG()` window.
- Include at least three DQ/reconciliation checks.
- Run `EXPLAIN` on at least two important queries and document observations.
- Recommend at least one index and one large-table partition strategy with justification.

